🤖 Interactive RAG — Docling + LangChain + FAISS

> **Reference paper:** *Attention Is All You Need* (Vaswani et al., 2017) — `1706.03762v7.pdf`

🧩 1. RAG 파이프라인 전체 개요

```
PDF
 │
 ▼
[Docling DocumentConverter]
 ├─ Text  ──► HybridChunker ──► text_chunks
 ├─ Table ──► export_to_markdown ──► table_chunks
 └─ Figure ──► caption + context ──► figure_chunks
                          │
                          ▼
              all_docs (text + table + figure)
                          │
                          ▼
           [OpenAI text-embedding-ada-002]
                          │
                          ▼
                FAISS Vector Store
                          │
                          ▼
              Retriever (MMR / Filtered)
                          │
             ┌────────────┘
             ▼
  [LCEL RAG Chain]
  question ──► retriever ──► format_docs ──► prompt ──► GPT-4o ──► answer
```

📦 2. Setup & 환경 설정

패키지 설치

```
pip install docling langchain langchain-openai langchain-community faiss-cpu python-dotenv jupyter
```

환경 변수 (`env.txt` → `.env`)

변수	값
`OPENAI_API_KEY`	OpenAI API 키
`LLM_MODEL`	`gpt-4o`
`EMBEDDING_MODEL`	`text-embedding-ada-002`
`LLM_BASE_URL`	(비워두면 OpenAI 기본값)
`EMBEDDING_BASE_URL`	(비워두면 OpenAI 기본값)

⚙️ 3. Configuration

```
PDF_PATH        = Path("1706.03762v7.pdf")
FAISS_INDEX_DIR = "faiss_nb_index"
EMBED_MODEL     = os.getenv("EMBEDDING_MODEL", "text-embedding-ada-002")
LLM_MODEL       = os.getenv("LLM_MODEL", "gpt-4o")

assert PDF_PATH.exists(), f"PDF not found: {PDF_PATH}"
```

📄 4. PDF 파싱 — `Docling`

Docling이 추출하는 요소

- 📝 **본문 텍스트** — 읽기 순서 보존
- 📋 **표(Table)** — 셀 구조 복원 → Markdown 변환
- 🖼️ **그림(Figure)** — 캡션 포함
- 🔠 **섹션 헤더** — 계층적 컨텍스트
- 📐 **수식(Formula)** — 구조 보존

`PdfPipelineOptions` 핵심 파라미터

파라미터	값	설명
`do_table_structure`	`True`	표 셀 구조 복원
`do_ocr`	`False`	텍스트 PDF → OCR 불필요

🗂️ 5. 메타데이터 스키마

> 모든 청크에 **동일한 스키마** 적용 → 필터 검색 + 출처 표시 + 컨텍스트 주입 가능

필드	타입	설명
`source`	str	PDF 파일명
`type`	str	`text` | `table` | `figure` | `formula` | `section_header`
`page`	int	1-indexed 페이지 번호
`section`	str	직속 부모 헤딩
`heading_path`	str	`/` -구분 조상 헤딩 경로
`caption`	str	표·그림 캡션 텍스트
`bbox`	str	`l,b,r,t` (PDF 포인트, 좌하단 원점)

> 💡 `heading_path` 를 **flat string**으로 저장하는 이유: FAISS 메타데이터 필터는 **스칼라** 값

✂️ 6. Text Chunking — `HybridChunker`

🔑 HybridChunker 특징

- ✅ **헤딩 경계** 유지 → 두 섹션에 걸친 청크 없음
- ✅ **토큰 예산** 내 분할 (기본: ST 토크나이저 max)
- ✅ 같은 섹션의 작은 청크 **자동 병합**
- ✅ `chunk.meta.headings` 에 **부모 헤딩 컨텍스트** 자동 포함

> vs 일반 `RecursiveCharacterTextSplitter`: 구조를 무시하고 문자 수 기반으로만 분할 → 섹션 혼재 가능

```
from docling.chunking import HybridChunker
from langchain_core.documents import Document as LCDocument

chunker = HybridChunker()
text_chunks: list[LCDocument] = []

for chunk in chunker.chunk(doc):
    if not chunk.text.strip():
        continue

    meta = chunk.meta
    headings: list[str] = meta.headings or []
```

📊 7. Table Extraction

전략

- 📌 표 1개 = 청크 1개
- **Markdown** 형식으로 export → 셀 구조 보존 + LLM 가독성
- `caption` → `page_content` 앞에 삽입 (임베딩 검색 가능) + `metadata['caption']` (UI 표시)

```
table_chunks: list[LCDocument] = []

for table in doc.tables:
    md_text = table.export_to_markdown(doc)
    caption = table.caption_text(doc) or ""

    page_no = 0
    bbox_str = ""
    if table.prov:
        prov = table.prov[0]
        page_no = int(prov.page_no)
        bb = prov.bbox
        bbox_str = f"{bb.l:.1f},{bb.b:.1f},{bb.r:.1f},{bb.t:.1f}"

    content_parts = []
    if caption:
        content_parts.append(caption)
```

🖼️ 8. Figure Extraction

전략

> 이미지 픽셀은 텍스트 벡터 DB에 저장 불가 → **텍스트 대리 신호** 사용

- 📌 **캡션** (가장 풍부한 의미 신호)
- 📌 **동일 페이지 텍스트** (최대 400자) — 주변 맥락 보강
- 예: "*encoder-decoder architecture*" 검색 → Figure 1 캡션 매칭

```
figure_chunks: list[LCDocument] = []

for pic in doc.pictures:
    caption = pic.caption_text(doc) or ""

    page_no = 0
    bbox_str = ""
    if pic.prov:
        prov = pic.prov[0]
        page_no = int(prov.page_no)
        bb = prov.bbox
        bbox_str = f"{bb.l:.1f},{bb.b:.1f},{bb.r:.1f},{bb.t:.1f}"

    page_texts = [
        c.page_content for c in text_chunks
        if c.metadata["page"] == page_no and c.metadata["type"] == "text"
    ]
    surrounding = page_texts[0][:400] if page_texts else ""

    content_parts = []
```

🗄️ 9. FAISS 인덱스 빌드

청크 통합 및 저장 구조

```
all_docs = text_chunks + table_chunks + figure_chunks
                ↓
    OpenAIEmbeddings (ada-002)
                ↓
    FAISS.from_documents()
                ↓
    faiss_nb_index/
      ├── index.faiss   ← 벡터 (float32)
      └── index.pkl     ← 메타데이터 dict (per-vector)
```

```python
from langchain_openai import OpenAIEmbeddings
from langchain_community.vectorstores import FAISS

all_docs = text_chunks + table_chunks + figure_chunks

embeddings = OpenAIEmbeddings(
    model=EMBED_MODEL,
```

🔍 10. Retriever 설정

MMR vs Filtered 비교

Retriever	`search_type`	언제 사용
MMR (Maximal Marginal Relevance)	`"mmr"`	기본 — 다양하고 중복 없는 청크
Filtered	`"similarity"`	타입/페이지/섹션 특정 시

MMR 동작 원리

```
fetch_k=20개 후보 검색 (유사도)
        ↓
k=5개로 재랭킹 (다양성 최대화)
        ↓
중복 청크 제거 효과
```

```
# 인덱스 로드
vectorstore = FAISS.load_local(
```

⛓️ 11. LCEL RAG Chain

파이프라인 구조

```
question ──► retriever ──► format_docs ──┐
                                         ├──► prompt ──► GPT-4o ──► answer
question ────────────────────────────────┘
```

`format_docs` — 컨텍스트 포맷터

- 각 청크 앞에 `[TYPE | page N | section: ... | caption: ...]` 헤더 삽입
- LLM이 "*Table 1 on page 6...*" 형태로 **출처 인용** 가능

System Prompt 핵심 지침

- 제공된 컨텍스트 청크만 사용
- TABLE 청크 → 직접 인용/요약
- FIGURE 청크 → 캡션으로 참조

💬 12. Demo Q&A

`ask()` 헬퍼 함수

- 체인 실행 + **소스 청크 출처 표시**
- `use_retriever` 파라미터로 리트리버 교체 가능

```
def ask(question: str, use_retriever=None) -> str:
    r = use_retriever or retriever
    print(f"Q: {question}")

    if use_retriever:
        chain = (
            {"context": r | format_docs, "question": RunnablePassthrough()}
            | prompt | llm | StrOutputParser()
        )
        answer = chain.invoke(question)
    else:
        answer = rag_chain.invoke(question)

    print(f"\nA: {answer}")
```

🚀 13. 고급: 메타데이터 기반 Retrieval 전략

A. 페이지 범위 필터

```
page_retriever = vectorstore.as_retriever(
    search_type="similarity",
    search_kwargs={"k": 5, "filter": {"page": 6}}
)
```

B. 섹션 범위 필터

```
section_retriever = vectorstore.as_retriever(
    search_type="similarity",
    search_kwargs={"k": 5, "filter": {"section": "Training"}}
)
```

C. 다중 타입 OR 검색 — `MergerRetriever`

> FAISS 필터: 단일 키 **exact match**만 지원 → OR 로직은 두 리트리버 병합

📌 정리

```
PDF 파싱 (Docling)
  └─ 구조 인식: 표·그림·수식·헤더 분리
        ↓
청킹 전략
  ├─ Text  → HybridChunker (헤딩 경계 + 토큰 예산)
  ├─ Table → Markdown export (1표 = 1청크)
  └─ Figure → caption + 주변 텍스트
        ↓
메타데이터 설계
  └─ type / page / section / heading_path / caption / bbox
        ↓
FAISS 인덱스 (ada-002 임베딩)
        ↓
MMR 리트리버 → 다양성 보장
        ↓
LCEL 체인 → GPT-4o → 출처 명시 답변
```